

Process Management and Exec Functions

Sistemas de Computadores

Departamento de Engenharia Informática

Instituto Superior de Engenharia do Porto

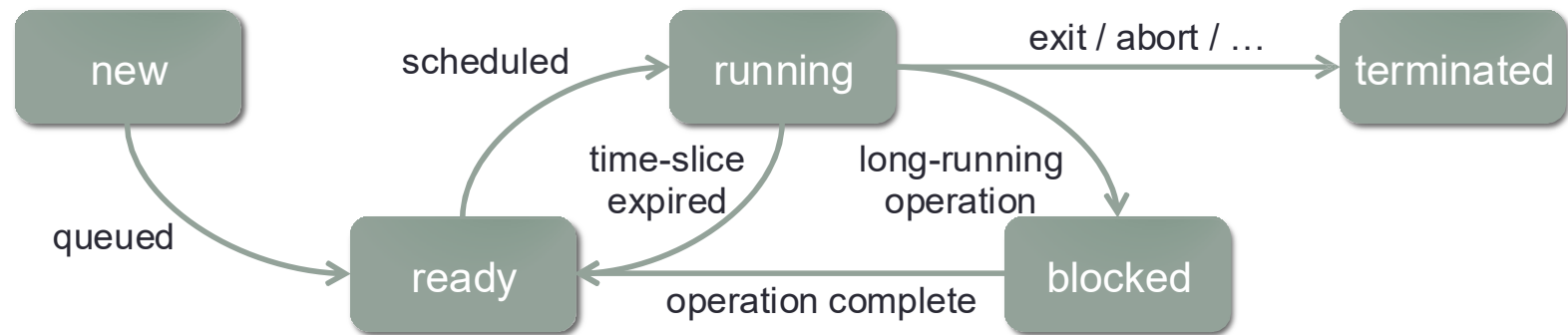
Luís Nogueira (lmn@isep.ipp.pt)

Recall from last class

- The child process is a duplicate of the parent, but with a unique Process ID (PID)
- Parent and child are independent processes that compete for CPU time
- After `fork()`, both processes execute the same code, starting from the instruction immediately after the `fork()` call
- The OS scheduler determines which process runs at any given moment, meaning their execution order is non-deterministic

States of a process

- A process can transition through multiple states during its lifetime
 - Such as new, ready, running (only one process per CPU core), waiting, and terminated, depending on the current activity and resource availability



Process termination

■ A process terminates:

- When it executes the last instruction
- Voluntary termination – calls `exit()`
- Abnormal termination – calls `abort()`
- Involuntary termination – killed by signal (`SIGSEGV`, `SIGKILL`, etc.)
- By action of the user or another process

■ When a process terminates, the parent is informed by the OS through the `SIGCHLD` signal

- This event is asynchronous – the parent can receive it at any time
- The parent can ignore the signal (the default behavior) or provide a function to be executed when the signal is received (a handler)
- (more on this in future classes)

The `waitpid` function

```
#include <sys/wait.h>

pid_t waitpid(pid_t pid, int *status, int options);
```

- **Blocks until the specified child process (given by `pid`) terminates**
 - `pid = -1`: Waits for any child process (similar to `wait()`)
- **The `options` parameter can be:**
 - `0` – Blocks until the child terminates
 - `WNOHANG` – Does not block; returns immediately if no child has exited
 - `WUNTRACED` – Also returns if a child is in the `STOPPED` state

The `waitpid` function

```
pid_t pid1, pid2;
int status;

pid1 = fork();

if(pid1 > 0){
    /* Parent waits for child to terminate */
    pid2 = waitpid(pid1, &status, 0);

    if(WIFEXITED(status))
        printf("Parent: child %d returned %d\n", pid2, WEXITSTATUS(status));
}
else{
    sleep(5);
    exit(10);
}
```

The `wait` and `waitpid` functions

- When a parent process calls `wait()`/`waitpid()`, one of the following may occur:
 - **Block** until a child process terminates
 - The parent is suspended until a child finishes execution
 - **Return immediately** if a child has already terminated
 - The exit status is collected without blocking
 - **Return immediately with an error** if there are no child processes
 - There is no child whose termination can be waited for

Zombie processes

- If the **child terminates before** the parent it becomes a **zombie**
 - Meaning it cannot execute, but the operating system still must retain information about its termination
- The OS cannot remove the process table entry until the parent collects the exit status
 - But can free all process resources
- Parent must acknowledge the termination for the entry to be removed
 - Acknowledgement is made with the `wait/waitpid` functions
 - After acknowledgement, the process disappears and is no longer a zombie

Orphan processes

- If the **parent terminates before** the child, the child become **orphan**
- In that case, it is necessary for the process to be adopted by another process to maintain hierarchy
- In UNIX-like OSes, orphan processes are adopted by the `init/ systemd` process
 - Which becomes their new parent and is responsible for reaping them when they terminate
 - In Linux, intermediate subreaper processes may adopt and reap orphaned children before they reach `systemd`

Why use `wait/waitpid` functions?

■ Resource management

- Reclaiming resources used by zombie processes prevents resource exhaustion and keeps the system running smoothly

■ Process management

- Knowing when child processes terminate allows for proper cleanup, error handling, and coordination between parent and child processes

■ System stability

- Preventing a buildup of zombie processes contributes to overall system stability and performance

Exercise 1

- **Implement a program that creates 10 child processes**
 - Each child sleeps 2 seconds more than its previous sibling
 - When ending, it must return as its exit value the number of seconds used for its sleeping time

- **The parent process must wait for all its children in the reverse order of process creation**
 - It should print both the PID and exit value of each ending process

exec () functions

- **Set of functions that allow for a process to execute a completely new program**
 - Provides the means for a process to change the program that it's executing as well as its entire memory image
- **The program being executed is replaced, but the process is still the same**
 - Same PID, same open files, ...
 - It's like a person and their job – when a person changes their job, they remain the same person, even though they're executing a new job

`exec ()` functions

■ Why are `exec ()` functions important?

- `exec ()` functions replace the current process image with a new one
- Unlike `fork ()`, which creates a new process, `exec ()` keeps the same PID but loads a different program
- It is fundamental in UNIX-based systems for process management

■ Typical uses of `exec ()` functions:

- Launching new programs
- Implementing shell commands
- Replacing a child process' code after `fork ()`
- Executing a different program within the same process

Key takeaway

- **`fork()` creates a new process**
- **`exec()` replaces the process image with a new program**
 - Open file descriptors remain open across `exec()`
 - Unless marked with `FD_CLOEXEC`
- **Together, they define both the process and the program it executes**
 - Every shell command typically uses `fork()` followed by `exec()`

Important!

- **The process does not return to the previous program because its code is entirely replaced**
- **The original program will only continue if the `exec ()` function encounters an error on its setup**
 - If it does return, something went wrong; the program that was supposed to be executed never ran, and `exec ()` returns -1
- **A successful execution of an `exec ()` function does not allow any return to the previous code**
 - Code after `exec ()` is executed only if `exec ()` fails

exec () functions

- **Although there are several different library functions in this `exec ()` family, they have one thing in common**
 - They change the program that the calling process executes to a program identified in their argument list
- **`execve ()` is the only real system call in the `exec` family**
 - All other `exec` functions (`execl ()`, `execv ()`, `execlp ()`, `execvp ()`, etc.) are library wrappers around `execve ()`
- **They differ in the way that the program and its arguments are passed to them**

The `exec1` and `execv` functions

```
#include <unistd.h>

int exec1(const char *path, const char *arg0, const char *arg1,
          ..., (char*)NULL);

int execv(const char *path, char *const args[]);
```

- **Replace the process's code with a program given by `path`**
 - **By convention**, `arg0` should have the name of the executable
- **Only one difference between `exec1()` and `execv()`**
 - `exec1()` receives the parameters with a **list of arguments**, ended with the NULL string
 - `execv()` receives arguments in a **vector of strings**; last position must have the NULL string

Example – `execl()`

- There's no need to check its return value since it must be -1
 - But the error code in `errno` will indicate what went wrong

```
#include <unistd.h>

int main(){
    int ret;

    /* arg0 is the command name */
    ret = execl("/bin/ls", "ls", "-l", (char*)NULL);

    /* The program only reaches this point if execl fails! */
    perror("execl");
    exit(EXIT_FAILURE);
}
```

Example – `execv()`

```
#include <unistd.h>

int main(){
    int ret;

    /* arg0 is the command name */
    char *cmd[] = {"ls", "-l", (char*)NULL};

    ret = execv("/bin/ls", cmd);

    /* The program only reaches this point if execv fails! */
    perror("execv");
    exit(EXIT_FAILURE);
}
```

The `execle` and `execve` functions

```
#include <unistd.h>

int execle(const char *path, const char *arg0, const char *arg1,
           ..., (char*)NULL, char *const envp[]);

int execve(const char *path, char *const args[],
           char *const envp[]);
```

■ Same behavior as previous functions

- `execve()` is what the kernel actually implements
- Other functions are convenience wrappers in `glibc`

■ Only adding the environment variables

- Using `char *const envp[]`

Example – `execle()`

```
#include <unistd.h>

int main(){
    int ret;

    char *env[] = {
        "LANG=C",          /* Force predictable, English output */
        "LC_ALL=C",        /* Disable locale-dependent formatting */
        "COLUMNS=40",      /* Affect ls output formatting */
    };

    ret = execle ("/bin/ls", "ls", "-l", (char *) NULL, env);

    /* The program only reaches this point if execle fails! */
    perror("execv");
    exit(EXIT_FAILURE);
}
```

The `exec1p` and `execvp` functions

```
#include <unistd.h>

int execlp(const char *path, const char *arg0, const char *arg1,
          ..., (char*)NULL);

int execvp(const char *path, char *const args[]);

int execvpe(const char *file, char *const argv[],
            char *const envp[]);
```

- **If the full path is not specified, the executable is searched in the folders provided in the `$PATH` environment variable**
 - `execvpe()` is a GNU/Linux extension, not part of POSIX; It is not portable to all UNIX systems (e.g., may not exist on BSD, macOS, etc.)

Example – `exec1p()`

```
#include <unistd.h>

int main(){
    int ret;

    /* arg0 is the command name */
    ret = exec1p("ls", "ls", "-l", (char*)NULL);

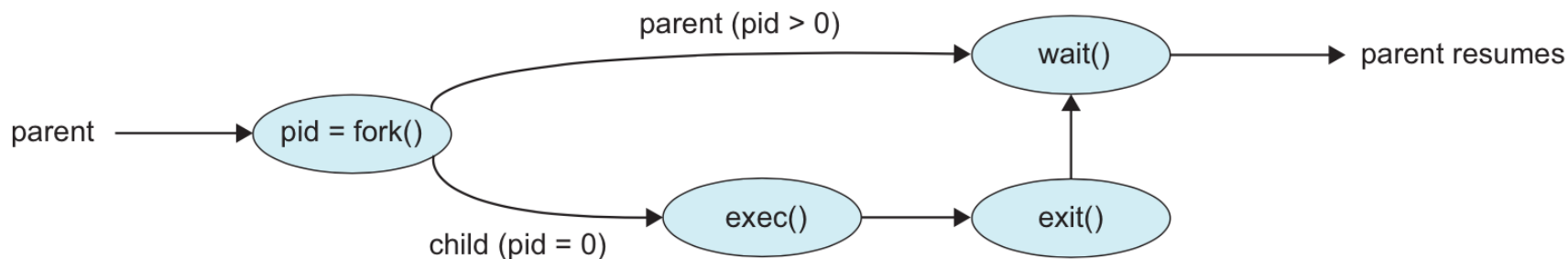
    /* The program only reaches this point if exec1p fails! */
    perror("exec1p");
    exit(EXIT_FAILURE);
}
```

The exec family of functions

Function name	Executable's specification	Argument specification	Environment
<code>execl()</code>	Pathname	List	Caller's environment variables
<code>execlp()</code>	Filename	List	Caller's environment variables
<code>execle()</code>	Pathname	List	<code>envp</code> argument
<code>execv()</code>	Pathname	Array	Caller's environment variables
<code>execve()</code>	Pathname	Array	<code>envp</code> argument (system call)
<code>execvp()</code>	Filename	Array	Caller's environment variables
<code>execvpe()</code>	Filename	Array	<code>envp</code> argument (GNU/Linux extension)

Typical use of exec functions

- A child process is created using `fork()`
- The `exec()` function is called in the child process to replace its image
- The parent process waits for the child's termination using `wait()` or `waitpid()`



Typical use of exec functions

```
#include <unistd.h>

pid = fork();

if(pid > 0){
    printf("Parent process (PID: %d) is waiting for child process
           to finish...\n", getpid());

    waitpid(pid, &status, 0);

    if(WIFEXITED(status))
        printf("Child process (exec) ended with exit value:
               %d\n", WEXITSTATUS(status));
    else
        printf("Child process terminated abnormally!\n");
}
else{
    printf("Child process (PID: %d) will execute a new
           program...\n", getpid());

    execlp("prog", "prog", (char*)NULL);
    exit(EXIT_FAILURE);
}
```

Exercise 2

- Consider the following excerpt from a program:

```
fork();  
fork();  
for(i=0; i<5; i++)  
    execlp("SCOMP", "SCOMP", (char*)NULL);
```

- How many times is the SCOMP program executed?
 - Justify your answer with a process tree

Exercise 3

- Consider the following excerpt from a program:

```
for(i=0; i<3; i++){  
    p = fork();  
    x = fork();  
  
    if(p>0 || x >0)  
        execlp("SCOMP", "SCOMP", (char*)NULL);  
}
```

- How many times is the SCOMP program executed?
 - Justify your answer with a process tree